Oscar Sorensen

Lenguajes de marcas

Tabla de contenidos

RA1. Reconoce las características de lenguajes de marcas analizando e interpretando fragmentos de código - explica qué funcion cumplen los lenguajes de marcas en tu proyecto.....2

RA2. Utiliza lenguajes de marcas para la transmisión y presentación de información a través de la web analizando la estructura de los documentos e identificando sus elementos. - Explica el uso de HTML y CSS que hace tu proyecto..... 3

RA3. Accede y manipula documentos web utilizando lenguajes de script de cliente. (Javascript)..... 4

RA6. Gestiona la información en formatos de intercambio de datos analizando y utilizando tecnologías de almacenamiento y lenguajes de consulta. (indica las consultas SQL de datos que utiliza tu aplicación e indica cómo se "pintan" los datos de esas consultas)..... 5

RA7. Opera sistemas empresariales de gestión de información realizando tareas de importación, integración, aseguramiento y extracción de la información. - Explica la aplicación empresarial de tu aplicación, o la relación de tu aplicación con el funcionamiento de una empresa..... 6

RA1. Reconoce las características de lenguajes de marcas analizando e interpretando fragmentos de código - explica qué funcion cumplen los lenguajes de marcas en tu proyecto

1_Screen Recording 2026-03-25 at 12.05.54.mov

Markup languages are used for the entire frontend of this project. HTML tags define the structure of the webpage and give us the meaning of the content. Without HTML and CSS, we would just have a white wall of unstyled text. Confusing and impractical. Markup languages are what organises, structures and shows the data in the way that we want to present and read it.

As always, I started with giving the webpage a header- this is where i tell the browser what charset, viewport and title to use, as well as giving it the link to the css stylesheet.

```
<!DOCTYPE html>

<html lang="en">

<head>

   <meta charset="UTF-8">

   <title><?php echo $title; ?></title>

   <meta name="viewport" content="width=device-width, initial-scale=1">

   <link rel="stylesheet" href="css/user.css">

</head>

<body>
```

Then I gave the page a body. This is where most of the action is happening. Here i have the different parts that i want to showcase, calls to action, different subjects and information for the user. This is set up in a way that is logical, intuitive and easy to navigate through. Html and CSS are core for that. What i showcase hereunder is just a snippet of the body. This part was many hundreds of lines long.

```
<div class="page">

   <header class="topbar">

       <div class="brand">
```

```
        <span class="dot"></span>
        <span>FirstListing</span>
    </div>
    <nav class="nav">
        <a href="how.php" lang-change="nav-how">How it works</a>
        <a href="helps.php" lang-change="nav-helps">Why it helps</a>
        <?php if (isset($_SESSION['user_id'])): ?>
            <a href="user.php" lang-change="nav-user">User</a>
            <a href="logout.php" lang-change="nav-logout">Logout</a>
        <?php else: ?>
            <a href="login.php" lang-change="nav-login">Login</a>
            <a href="register.php" lang-change="nav-register">Register</a>
        <?php endif; ?>
        <button id="lang-toggle" class="lang-btn">ES</button>
        <a href="admin/admin.php" class="pill">Admin</a>
    </nav>
</header>
```

The site also has a footer. This is at the bottom of the page and showcases relevant information, name of company, as well as legal and privacy notices. Again html and css are used to style it.

```
<footer class="footer">
    <span lang-change="footer-mvp">FirstListing — School MVP</span>
    <div class="footer-links">
        <a href="privacy.php" lang-change="footer-privacy">Privacy Policy</a>
        <a href="legal.php" lang-change="footer-legal">Legal Notice</a>
    </div>
```

```
    </footer>
```

Markup languages are used throughout the whole project. It is both used to give style and present the data properly, but html is also used to organise the data with tags. When every line has a html tag it is easier to locate it with css, and apply style to specific areas. This helps to keep everything organised and easy to manage.

Without markup languages, the overall experience of the website would be considerably worse. This matters both to the end user and the administrators of the website.

GitHub link

https://github.com/oscarsorensen/FirstListing/blob/main/public/index.php

RA2. Utiliza lenguajes de marcas para la transmisión y presentación de información a través de la web analizando la estructura de los documentos e identificando sus elementos. - Explica el uso de HTML y CSS que hace tu proyecto

2_Screen Recording 2026-03-25 at 12.22.42.mov

The way markup languages are used in the project goes from starting with html, moving to css and then ending at the webpage.

We can follow the different parts of the code and how they connect, if we look throughout the code. In the video shows we start with a layout of the user.php page, where i show a html block. here we can see how html is used to tag a part of the website. The html part is long, running from from line 184 to 438, which is why this is just a snippet.

```
<section class="user-layout">

    <div class="user-main">


        <?php foreach ($errors as $err): ?>

            <div class="error"><?= esc($err) ?></div>

        <?php endforeach; ?>


        <?php if ($already_in_db && $raw_page): ?>

            <div class="notice-already-seen">

                This URL is already in our database — first crawled on

                    <strong><?= esc((string)($raw_page['first_seen_at'] ?? '—'))
?></strong>.

                  Showing existing data and searching for cross-portal duplicates
below.

            </div>
```

```
            <?php endif; ?>



            <!-- URL input form -->

            <div class="tool-card user-tool-card">

                    <div class="card-topline" lang-change="usr-card1-top">Duplicate
Check</div>

                <h2 lang-change="usr-card1-h2">Paste listing URL</h2>

                    <p class="muted-sm" lang-change="usr-card1-sub">Paste a property
URL. The check takes about 10-20 seconds.</p>

                <form method="post" action="user.php" class="user-search-form">

                    <div class="field">

                        <label for="listing_url" lang-change="usr-label-url">Listing
URL</label>

                        <input id="listing_url" name="listing_url" type="url"

                               placeholder="https://example.com/listing/123"

                               value="<?= esc($input_url) ?>">

                    </div>

                    <div class="actions">

                                                <button class="btn" type="submit"
lang-change="usr-btn-check">Check duplicates</button>

                                        <a class="ghost ghost-light" href="index.php"
lang-change="usr-btn-back">Back to homepage</a>

                    </div>

                </form>

            </div>
```

However, moving on the css we can see how the html tags are then used in the css, to select the specific parts of the website we want. Now from there, it is possible to the css to style the different organised parts of the html.

```
/* User page (MVP frontend) */
```

```
.user-layout {
    display: grid;
    grid-template-columns: minmax(0, 2fr) minmax(0, 1fr);
    gap: 18px;
    align-items: start;
}
@media (max-width: 900px) {
    .hero {
        grid-template-columns: 1fr;
        padding: 18px;
        min-height: auto;
    }
    .hero-text h1 { font-size: 42px; }
    .hero-side { grid-template-columns: 1fr; }
    .page-hero-small { grid-template-columns: 1fr; }
    .grid { grid-template-columns: 1fr; }
    .trust { grid-template-columns: 1fr; }
    .mini { grid-template-columns: 1fr; }
    .final-cta-card {
        flex-direction: column;
        align-items: flex-start;
    }
    .user-layout {
        grid-template-columns: 1fr;
    }
    .nav {
        width: 100%;
        justify-content: flex-start;
```

```
        gap: 8px;

    }

    .topbar {

        flex-direction: column;

        align-items: flex-start;

    }

}
```

Another way css has been used is with the :root function. What I have done here is make custom properties. It's a bit more advanced than just styling specific blocks. With the root I store spacing as variables and make them changeable in one place. It makes it much easier to change and style generally later in the project.

The class names in the HTML (tool-card, state-pill, score-badge) connect directly to rules in the CSS file.

```
:root {

    --bg: #e5eef8;

    --bg-2: #d4e1f1;

    --card: #ffffff;

    --card-soft: #f2f6fb;

    --text: #0f1a2b;

    --muted: #5d6d86;

    --border: #c9d6ea;

    --accent: #2563eb;

    --accent-2: #f59e0b;

    --shadow: 0 16px 34px rgba(24, 39, 64, 0.14);

    --shadow-strong: 0 22px 60px rgba(17, 27, 45, 0.22);

}
```

All in all, the html and css form an integral part of the project. The markup languages are used to structure documents by giving them ids and then identifying elements by those ids- so that we can style and change specific items. I have here shown how both general style can be applied, and specific. All of these things are especially important for the non-technical end user, who would not have been able to navigate as easily or cleanly, without the style that markup languages gives us.

GitHub link

https://github.com/oscarsorensen/FirstListing/blob/main/public/css/user.css

RA3. Accede y manipula documentos web utilizando lenguajes de script de cliente. (Javascript)

3_Screen Recording 2026-03-25 at 12.37.29.mov

Another markup language that i havnt talked about yet is JavaScript. Javascript is more for scripting than for styling, and therefore made, to be able to make other things happen. This can be searches on the website, small functions or things that change. Basically javascript is a language for manipulating web documents in the client-side.

In this project I made a toggle that could switch the language of the website. What i did was, since i have hardcoding the english descriptions in the html, i gave each of the pieces of text in english, a tag. Then i took that tag in javascript, and made a script to that if the ES/EN button is clicked, we switch out the original english hardcoded html, with a specific spanish part. I made a file called lang.js for that.

```
// Spanish translations — key is a short ID, value is the Spanish text.

// English text lives in the HTML itself (not here).

var es = {

   // Navigation (shared across all pages)

   "nav-how":      "Cómo funciona",

   "nav-helps":    "Por qué ayuda",

   "nav-login":    "Iniciar sesión",

   "nav-register": "Registrarse",

   "nav-home":     "Inicio",

   "nav-logout":   "Cerrar sesión",

   "nav-user":     "Usuario",

   "back-home":    "Volver al inicio",



   // index.php

   "idx-eyebrow":      "Proyecto escolar · Detección de duplicados inmobiliarios",

    "idx-h1":               "Rastrea anuncios duplicados y compara quién apareció
primero.",
```

```
    "idx-lead":            "FirstListing guarda evidencia bruta de anuncios (HTML,
texto y JSON-LD)

................
```

At the end of the file I have the actual script. I call the function applyLang, because it doesn't change the hardcoded english language, it just applies the spanish language instead. This means “show instead of”, not “change with”. The base language is still English. It works by looping over all elements with a lang-change tag, looks up the key in es, and sets the text. When switching back to English it restores from a data-en attribute that was saved on page load by saveEnText().

```
// Swap text on all translatable elements to the chosen language
function applyLang(lang) {
   document.querySelectorAll('[lang-change]').forEach(function(el) {
       var key = el.getAttribute('lang-change')
       if (lang === 'es' && es[key]) {
           el.textContent = es[key]
       } else {
           // Restore the original English text from the snapshot
           el.textContent = el.getAttribute('data-en')
       }
   })

   // Show the OTHER language on the toggle button
   var btn = document.getElementById('lang-toggle')
   if (btn) {
       btn.textContent = lang === 'es' ? 'EN' : 'ES'
   }
}
```

Below the language change script I have an eventListener that ties the different parts together. It runs every time a webpage is accessed and reads the saved language from localStorage. Without this part, you would “reset” the language every time you changed pages, because the program wouldn't know which language you were on. In that case you would start with English on every page, and have to change it to Spanish manually.

```
document.addEventListener('DOMContentLoaded', function() {
   // Step 1: snapshot English text (must happen before any swap)
   saveEnText()

   // Step 2: apply saved language
   applyLang(currentLang)

   // Toggle button switches language and saves preference
   var btn = document.getElementById('lang-toggle')
   if (btn) {
       btn.addEventListener('click', function() {
           currentLang = currentLang === 'en' ? 'es' : 'en'
           localStorage.setItem('lang', currentLang)
           applyLang(currentLang)
       })
   }
})
```

Overall this ties into convenience for the end user. With JavaScript it is possible to change languages, as well as remembering the languages across both pages and sessions. This would now have been possible with only html and css, which makes javaScript a uniquely important language when programming webpages.

GitHub link

https://github.com/oscarsorensen/FirstListing/blob/main/public/js/lang.js

RA6. Gestiona la información en formatos de intercambio de datos analizando y utilizando tecnologías de almacenamiento y lenguajes de consulta. (indica las consultas SQL de datos que utiliza tu aplicación e indica cómo se "pintan" los datos de esas consultas)

6_Screen Recording 2026-03-25 at 12.55.49.mov

One of the ways that the program handles data flow in different formats, happens when a URL crawl request is being petitioned. This starts in user.php, but quickly activates find_duplicates.php. In find_duplicates.php a select is happening on the sql data, whereafter it is converted to a JSON string. We do this so PHP can capture the string as the commands output.

```
$sql = "

   SELECT

       ai.id,

       ai.raw_page_id,

       ai.title,

       ai.price,

       ai.sqm,

       ai.rooms,

       ai.bathrooms,

       ai.property_type,

       ai.listing_type,

       ai.address,

       ai.reference_id,

       ai.description,

       rp.url,

       rp.domain,
```

```
       rp.first_seen_at,

       (

                   IF(ai.reference_id  IS  NOT  NULL  AND  ai.reference_id  !=  ''  AND
ai.reference_id = :ref,   5, 0) +

               IF(ai.price           IS  NOT  NULL  AND  ai.price          = :price,
3, 0) +

               IF(ai.sqm              IS  NOT  NULL  AND  ai.sqm             = :sqm,
3, 0) +

               IF(ai.rooms           IS  NOT  NULL  AND  ai.rooms          = :rooms,
2, 0) +

               IF(ai.bathrooms     IS NOT NULL AND ai.bathrooms     = :bathrooms,
2, 0) +

            IF(ai.property_type IS NOT NULL AND ai.property_type = :property_type,
1, 0) +

             IF(ai.listing_type  IS NOT NULL AND ai.listing_type  = :listing_type,
1, 0)

       ) AS match_score

   FROM ai_listings ai

   JOIN raw_pages rp ON rp.id = ai.raw_page_id

   WHERE ai.raw_page_id != :exclude

   HAVING match_score >= 5

   ORDER BY match_score DESC

   LIMIT 20

";
```

```
echo json_encode($candidates, JSON_UNESCAPED_UNICODE | JSON_PRETTY_PRINT);
```

Then again in user.php, we can decode this json string back into a PHP array. This json code and decode needs to happen because it is otherwise it wouldn't be able to move data across the different files like that. There are other ways to do it, but this is the most simple.

```
                $decoded = json_decode(implode('', $dupes_out), true);

                if (is_array($decoded) && !isset($decoded['error'])) {

                    $candidates = $decoded;

                }

                $status_matches = count($candidates);
```

When we now have the PHP array, we can loop over it and build a HTML table, which is what we really wanted to do in the first place.

```
<?php if (!empty($candidates)): ?>

                                <div class="table-wrap" style="max-height: 460px;
overflow:auto;">

                        <table>

                            <tr>

                                        <th lang-change="usr-th-score">Score</th><th
lang-change="usr-th-domain">Domain</th><th lang-change="usr-th-title">Title</th>

                                        <th lang-change="usr-th-price">Price</th><th
lang-change="usr-th-sqm">SQM</th><th lang-change="usr-th-rooms">Rooms</th>

                                            <th lang-change="usr-th-firstseen">First
seen</th><th                lang-change="usr-th-ai">AI                result</th><th
lang-change="usr-th-url">URL</th>

                            </tr>

                            <?php foreach ($candidates as $c):

                                $score         = (int)$c['match_score'];

                                [$label, $css] = score_info($score);

                                                        $ai                     =
$ai_comparisons[(int)$c['raw_page_id']] ?? null;

                            ?>

                            <tr>

                                        <td><span class="score-badge <?= $css ?>"><?=
esc($label) ?> (<?= $score ?>)</span></td>

                                <td><?= esc((string)($c['domain'] ?? '—')) ?></td>
```

```
                                <td><?= esc((string)($c['title'] ?? '—')) ?></td>

                                        <td><?= fmt_price($c['price'] !== null ?
(int)$c['price'] : null) ?></td>

                                  <td><?= $c['sqm'] !== null ? esc((string)$c['sqm'])
. ' m²' : '—' ?></td>

                                                  <td><?= $c['rooms'] !== null ?
esc((string)$c['rooms']) : '—' ?></td>

                                    <td><?= esc((string)($c['first_seen_at'] ?? '—'))
?></td>

                                <td><?= render_ai_badge($ai) ?></td>

                                <td>

                                     <a class="link" href="<?= esc((string)($c['url']
?? '#')) ?>" target="_blank" rel="noopener">

                                         <?= esc(parse_url((string)($c['url'] ?? ''),
PHP_URL_HOST) ?: $c['url'] ?? '—') ?>

                                     </a>

                                </td>

                            </tr>

                            <?php endforeach; ?>

                        </table>

                    </div>

                        <p class="hint" lang-change="usr-hint-score">Score = sum of
matching fields (max 17). Hover AI badge for reason.</p>


                <?php elseif ($submitted && $status_parsed): ?>

                        <p class="muted-sm" lang-change="usr-no-dupes">No candidates
scored 10 or higher. The listing may be unique in our database.</p>


                <?php else: ?>

                    <p class="hint" lang-change="usr-hint-submit">Submit a URL above
to see duplicate candidates here.</p>
```

```
            <?php endif; ?>

        </div>
```

Now to summarise, the chain goes like this:

SQL rows → json_encode → text → json_decode → PHP array → HTML table → browser.

All of this is to make the outputs clear and understandable, as well as easy to handle by the program. The end result is a much nicer user experience, now that the data is organised. Throughout the flow, the SQL queries are formatted, data is exchanged as well as stored in and over different types of technologies. This is done so the data can be properly analysed and used.

Because of this we can conclude that all of the different markup and query languages are necessary, as well as the ability to use and understand data in different formats, and how it is exchanged and stored.

GitHub link

https://github.com/oscarsorensen/FirstListing/blob/main/scripts/find_duplicates.php

RA7. Opera sistemas empresariales de gestión de información realizando tareas de importación, integración, aseguramiento y extracción de la información. - Explica la aplicación empresarial de tu aplicación, o la relación de tu aplicación con el funcionamiento de una empresa.

7_Screen Recording 2026-03-25 at 13.16.12.mov

Now, the business problem is very straightforward. The same property appears on multiple portals listed by different agents, and there is no easy way to know who published it first. FirstListing crawls portals, timestamps them, and lets an agent or user, check whether a listing already exists in the database and who had it first.

This is extremely practical, because if you can crawl all, or most sites, automatically, every hour, you can build a giant database over what is for sale and when it came for sale. If you can find the same ad on 5 different websites, and you can see that it appeared on one specific website 4 hours before the others, it is easy to conclude that that is the original agent.

If this could go live, it would majorly decrease the time agents waste on trying to find the original publisher, as well as do it more securely. If you could just skip a middleman a single time, it is possible to save thousands of euros that you wouldn't have to share in agent fees.

So to re-iterate, the crawler fetches the URL, AI parser extracts structured fields, SQL scoring finds candidates, AI comparison checks descriptions. We are presented with the top 20 candidates and the top 5 get AI comparisons on the descriptions, to try to figure out if they really are the same. The end result is an overview over similar, or the same ads. Then we can find out who the original publisher is, and contact those directly. It is both great for saving time and money.

GitHub link

https://github.com/oscarsorensen/FirstListing/blob/main/public/user.php